

Curso Libre de Configuración 2: Análisis de Datos

Tarea 2: Números Pseudoaleatorios y Métodos de Monte Carlo

Facultad de Ingeniería, Universidad del Istmo de Guatemala

Fecha de entrega: 19 de septiembre 2025, 11:59PM

1 Introducción

En esta ocasión haremos una pequeña inmersión en dos mundos: el de los números pseudoaleatorios y el de las simulaciones de Monte Carlo. Los números pseudoaleatorios son esenciales en el estudio de simulaciones de todo tipo, en videojuegos y en criptografía.

Como todos (espero) sabrán, las computadoras no son capaces de generar números completamente aleatorios. Si estamos hablando de un solo número, un número aleatorio es aquel tomado de un conjunto de posibles valores, cada uno con una probabilidad equitativa, es decir, como en una distribución uniforme $X \sim \text{Unif}(a, b)$. Cuando hablamos de secuencias de números aleatorios, tenemos que asegurarnos además de que estos sean *independientes* uno del otro.

Los generadores de números pseudoaleatorios, o PRNGs por sus siglas en inglés, utilizan algoritmos o tablas precalculadas para generar secuencias de números que parecen ser aleatorios. La diferencia entre un PRNG y un TRNG, o un generador de verdaderos aleatorios, puede explicarse como los lanzamientos de un dado. Si utilizamos un PRNG, entonces estamos obteniendo una lista de los lanzamientos, y solo estamos preguntando cuál es el siguiente número en la lista, mientras que un TRNG hace que la computadora “verdaderamente” lance el dado. Los PRNGs son *eficientes*, ya que pueden producir una gran cantidad de números en una cantidad pequeña de tiempo, y también son *deterministas*, lo cual quiere decir que una secuencia en particular puede volverse a producir en cualquier momento si se sabe su punto de partida. A su vez, también son *periódicos*, es decir, que la secuencia se repite después de cierto tiempo. Aunque esta no es una característica deseada en un generador de números aleatorios, la mayoría de PRNGs modernos tienen períodos tan largos (hasta $2^{31} - 1$ números antes de que la secuencia se repita) que para efectos prácticos no importa [2].

¿Y esto para qué sirve?

Hay cantidades que son muy fáciles de *definir* pero muy difíciles, o de prácticamente imposibles, de *calcular* de forma exacta. Piensen en una integral que no tiene antiderivada elemental, en el volumen de una región en 20 dimensiones, en la probabilidad de que un sistema complejo falle, o en el valor esperado de una estrategia dentro de un juego. En todos esos casos la respuesta existe, pero no hay una fórmula cerrada que nos la dé, y los métodos determinísticos que ya conocen o bien no aplican, o se vuelven prohibitivamente caros computacionalmente hablando.

La idea de Monte Carlo es cambiar el cálculo exacto por una estimación. Si logramos escribir la cantidad que buscamos como el valor esperado $\mathbb{E}[g(X)]$ de alguna variable aleatoria, entonces podemos muestrear realizaciones de X (usando nuestros PRNGs) y obtener un promedio. La Ley de los Grandes Números nos garantiza que ese promedio converge al valor real, y el Teorema del Límite Central nos dice qué tan rápido lo hace.

Con esta tarea quiero que vean tanto lo que esta idea nos permite hacer como el punto donde se rompe. Por un lado, la calidad del estimador depende directamente de la calidad del PRNG que lo alimenta, y eso lo van a comprobar ustedes mismos con el caso RANDU. Por el otro, el error decae de una forma lenta, pero que no empeora conforme crece la dimensión del problema, que es justamente lo que vuelve a Monte Carlo tan útil en problemas donde los demás métodos se quedan cortos.

2 Investigación – Primera parte: Generadores

Nuestra inmersión en el mundo de los números pseudoaleatorios consistirá en investigar acerca de distintos algoritmos generadores de pseudoaleatorios e implementarlos en `Python`. Los algoritmos que deberán investigar e implementar son los siguientes:

1. Generadores de congruencia lineal (LCG por sus siglas en inglés)
2. Método de cuadrados medios o middle-square method
3. Mersenne Twister
4. Blum Blum Shub
5. RANDU

Para cada algoritmo deberán investigar y entregar lo siguiente:

1. Pseudocódigo y fundamento matemático de cada algoritmo.
2. Tabla comparando la velocidad, período, seguridad y casos de uso de cada algoritmo.
3. Implementación de cada algoritmo a partir del pseudocódigo en `Python` (**desde cero; no se permite usar `numpy.random` ni `random` como el motor del generador**).

- Para cada implementación, deberán incluir lo siguiente:
 - (a) Histograma mostrando los números generados (normalizados a $[0, 1]$).
 - (b) Pruebas de hipótesis para comprobar uniformidad e independencia de las secuencias generadas. Como mínimo necesito dos: una prueba de bondad de ajuste para uniformidad (e.g., chi-cuadrado o Kolmogórov–Smirnov) y una prueba de independencia serial (e.g., autocorrelación de retardo 1 o una prueba de rachas). Para cada una reporten el estadístico, el p -valor y qué significa ese resultado, en una o dos oraciones.
 - (c) *Prueba espectral visual*: para al menos LCG y RANDU, grafiquen ternas consecutivas (u_i, u_{i+1}, u_{i+2}) como puntos en el cubo $[0, 1]^3$. Comenten qué observan y relaciónenlo con el período y la calidad de cada generador.
 - (d) Comparación de un algoritmo a su elección con las librerías estándar `random` y `secrets` de Python.

Reproducibilidad. Todo generador debe recibir su semilla como argumento y documentarla. Fijen y reporten las semillas que usaron en cada figura, para que cualquiera pueda correr su código y obtener el mismo resultado.

3 Investigación – Segunda Parte: Monte Carlo

Una de las aplicaciones más conocidas de los números pseudoaleatorios son los métodos de Monte Carlo. Los métodos de Monte Carlo se refieren a una amplia clase de algoritmos que utilizan el muestreo de números aleatorios para resolver problemas que son determinísticos en naturaleza, como optimización, aproximar integrales o simular distribuciones de probabilidad.

3.1 Fundamento: el estimador y su error

Antes de ponerse a programar, quiero que deriven a mano los siguientes resultados, en su documento y con su propia notación. Citarlos no cuenta:

1. Muestren que la integral $I = \int_a^b f(x) dx$ puede escribirse como $I = (b - a) \mathbb{E}[f(U)]$ con $U \sim \text{Unif}(a, b)$, y que por lo tanto el estimador:

$$\hat{I}_N = \frac{b - a}{N} \sum_{i=1}^N f(U_i)$$

es *insesgado*, es decir, que $\mathbb{E}[\hat{I}_N] = I$.

2. Deriven la varianza $\text{Var}(\hat{I}_N)$ y muestren que el error estándar del estimador decae como $\mathcal{O}(N^{-1/2})$. Expliquen con sus propias palabras por qué esta tasa no depende de la dimensión del problema.

3. Construyan un intervalo de confianza del 95% para $\hat{I}_N$ usando el Teorema del Límite Central. De aquí en adelante, toda estimación numérica que reporten tiene que venir acompañada de su intervalo de confianza.

3.2 Calentamiento: integrales en una dimensión

Utilicen su estimador de Monte Carlo con realizaciones de $X \sim \text{Unif}(a, b)$ para aproximar:

$$(a) \quad \int_0^1 \sin(\pi x) \, dx \qquad (b) \quad \int_0^2 \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \, dx$$

Para cada una:

1. Reporten la estimación con su intervalo de confianza del 95% y compárenla contra el valor teórico real.
2. **Estudio de convergencia.** Grafiquen el error absoluto $|\hat{I}_N - I|$ contra N en escala log-log, para N desde 10^1 hasta al menos 10^6 . Ajusten una recta y estimen la pendiente. ¿Les cuadra con la tasa $\mathcal{O}(N^{-1/2})$ que derivaron? Si algo se sale de lo esperado, coméntenlo.
3. Repitan el cálculo usando (i) un generador estándar de `Python` y (ii) uno de los PRNGs que implementaron en la Parte 1. Comparen sus resultados.

3.3 Alta dimensión y la maldición de la dimensionalidad

Hasta aquí Monte Carlo se ve como una manera cara de calcular integrales que ya sabíamos resolver. Esta sección es donde eso cambia. Consideren el problema de calcular el volumen de la bola unitaria en d dimensiones,

$$V_d = \text{Vol}(\{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\|_2 \leq 1\}) = \int_{[-1,1]^d} \mathbf{1}[x_1^2 + \dots + x_d^2 \leq 1] \, d\mathbf{x},$$

que tiene el valor cerrado conocido $V_d = \pi^{d/2} / \Gamma(\frac{d}{2} + 1)$. Ese valor úsenlo únicamente como *verdad de referencia*, no como parte del estimador.

1. Implementen un estimador de Monte Carlo para V_d muestreando puntos uniformes en el cubo $[-1, 1]^d$ y contando la fracción que cae dentro de la bola. Reporten la estimación y su intervalo de confianza para $d = 2, 5, 10, 20$.
2. **Comparación contra cuadratura determinística.** Implementen también una estimación por rejilla, dividiendo cada eje en m subintervalos y evaluando el integrando en el centro de cada celda. Para un mismo presupuesto de puntos, comparen el error de Monte Carlo contra el de la rejilla conforme d crece. Adicionalmente, construyan una tabla con el número de puntos que necesitaría la rejilla para tener $m = 10$ subdivisiones por eje en $d = 2, 5, 10, 20$ dimensiones. ¿Qué observan? Relacionen esto con el término “maldición de la dimensionalidad”.

3. Grafiquen la fracción del volumen del cubo que ocupa la bola, $V_d/2^d$, como función de d . Expliquen qué fenómeno geométrico revela esa gráfica y qué implica a la hora de diseñar un buen estimador en alta dimensión. En particular, ¿por qué el muestreo uniforme en el cubo se vuelve tan ineficiente?

3.4 Reducción de varianza

El error de Monte Carlo decae como $\sigma/\sqrt{N}$, así que solo hay dos formas de bajarlo: subir N , que es caro, o bajar σ , que es mucho más interesante. Investiguen e implementen al menos dos de las siguientes técnicas, aplicándolas a la integral (a) o (b) de la sección 3.2:

- Variables antitéticas (antithetic variates)
- Muestreo por importancia (importance sampling)
- Variables de control (control variates)

Para cada técnica expliquen el fundamento matemático, impleméntenla y cuantifiquen la reducción de varianza que lograron. Reporten el factor de reducción, es decir, la razón de varianzas respecto a Monte Carlo simple con el mismo N , y el “speedup efectivo” equivalente en número de muestras. ¿La técnica siempre ayuda o depende del integrando? Justifiquen su respuesta.

3.5 Cuando el generador arruina el resultado: el caso RANDU

RANDU es un generador históricamente famoso por lo malo que resultó ser: sus ternas consecutivas caen sobre un número pequeño de hiperplanos dentro del cubo $[0, 1]^3$ [1]. En esta sección van a reproducir esa falla y diagnosticarla.

1. Usen su implementación de RANDU como fuente de aleatoriedad para estimar por Monte Carlo alguna cantidad que dependa de verdad de ternas (x, y, z) en 3D. Por ejemplo, el volumen de la bola unitaria en $d = 3$ o la integral $\iiint_{[0,1]^3} \mathbf{1}[x^2 + y^2 + z^2 \leq 1] dx dy dz$.
2. Repitan la misma estimación usando el Mersenne Twister (o `random` estándar).
3. Comparen ambos estimadores contra el valor real. Documenten la diferencia y, apoyándose en la gráfica de ternas en 3D, expliquen geométricamente por qué RANDU produce un sesgo o una convergencia rara mientras que el buen generador no. Conecten esto explícitamente con la prueba espectral de la Parte 1.

La moraleja que quiero que articulen en su análisis es sencilla: un método de Monte Carlo nunca va a ser mejor que su fuente de aleatoriedad.

4 Entregables

A Blackboard deberán subir únicamente el link a un repositorio de GitHub conteniendo sus soluciones. En el repositorio deberán incluir lo siguiente:

- Documento en `.pdf` y código `.tex` con su investigación de las partes 1 y 2 y el análisis de los resultados obtenidos. El documento debe contener las derivaciones a mano de la sección 3.1, todas las figuras (histogramas, pruebas espectrales, gráficas de convergencia log-log, gráfica de $V_d/2^d$), las tablas comparativas y los intervalos de confianza de cada estimación.
- Archivos `.py` con las implementaciones de los PRNGs y de los métodos de Monte Carlo. El código debe correr de principio a fin y reproducir las figuras del documento a partir de las semillas documentadas.
- Un `README.md` corto explicando cómo ejecutar el código.

Recordatorio

La política de entregas tarde indica que tienen 2 días de gracia para entregar con el 80% de la nota y 50% después de este período. **El uso indebido e irresponsable de asistentes de IA/LLMs resultará en 0 y reporte con el Consejo de Facultad.** Recuerden que las derivaciones a mano, el estudio de convergencia con sus propias semillas y el análisis interpretativo son individuales: deben poder explicarme y defender oralmente cualquier parte de su entrega.

Referencias

- [1] George Marsaglia. “Random Numbers Fall Mainly in the Planes”. In: *Proceedings of the National Academy of Sciences* 61.1 (1968), pp. 25–28. DOI: 10.1073/pnas.61.1.25.
- [2] RANDOM.ORG. *Introduction to Randomness and Random Numbers*. URL: <https://www.random.org/randomness/> (visited on 09/01/2025).